

Introducing a name for brace-or-equal-initializers for non-static data members

ISO/IEC JTC1 SC22 WG21
P0134R0
Richard Smith
richard@metafoo.co.uk
2015-09-23

N2756 introduced the ability to specify a default initializer for a non-static data member:

```
class X {  
    int a = 123;  
    X() {} // initialize 'a' to 123  
};
```

However, it did not introduce a useful term into the standard to describe these entities. This has resulted in a number of terms being in common usage to describe these initializers:

- **Non-static data member initializers:** This is the name of the papers introducing the feature. It is also the name used by GCC, where it is frequently shortened to "NSDMI" due to its unwieldy length.
- ***brace-or-equal-initializer for a non-static data member*:** This is the name used by the standard.
- **In-class initializers**
- **Default member initializers**

As Project Editor, I believe that having a name for this construct is useful for the presentation of the C++ standard, and that having a single name is useful for discourse within the C++ community at large. As with the term "forwarding reference", agreeing on the term within the C++ committee and using the term within the C++ standard is the best way to get a single term adopted.

Based on many discussions within and outside the committee, this paper proposes that we use the term *default member initializer* for this construct. This term is specific enough to uniquely identify the scenario, reflects the symmetry between this construct and similar cases (default arguments and default template arguments, and a constructor's *mem-initializers*), and is terse enough that an abbreviation (such as NSDMI) does not need to be invented.

Proposed wording

Change in [expr.prim.general] (5.1.1) paragraph 4:

Otherwise, if a member-declarator declares a non-static data member (9.2) of a class X, the expression this is a prvalue of type "pointer to X" within the optional ~~brace-or-equal-initializer~~ **default member initializer**. It shall not appear elsewhere in the member-declarator.

Change in [expr.prim.general] (5.1.1) paragraph 5 example:

```
class Outer {  
    int a[sizeof(*this)];           // error: not inside a member function  
    unsigned int sz = sizeof(*this); // OK: in brace-or-equal-initializer default member initializer  
    [...]
```

Change in [dcl.init.aggr] (8.5.1) paragraph 7:

If there are fewer initializer-clauses in the list than there are members in the aggregate, then each member not explicitly initialized shall be initialized from its ~~brace-or-equal-initializer~~ **default member initializer (9.2)** or, if there is no ~~brace-or-equal-initializer~~ **default member initializer**, from an empty initializer list (8.5.4).

Change in [dcl.init.aggr] (8.5.1) paragraph 8:

If a reference member is initialized from its ~~brace-or-equal-initializer~~ **default member initializer** and a potentially-evaluated subexpression thereof is an aggregate initialization that would use that ~~brace-or-equal-initializer~~ **default member initializer**, the program is ill-formed.

Change in [class.mem] (9.2) paragraph 2:

Within the class member-specification, the class is regarded as complete within function bodies, default arguments, using-declarations introducing inheriting constructors (12.9), exception-specifications, and ~~brace-or-equal-initializers for non-static data members~~ **default member initializers** (including such things in nested classes)

Change in [class.mem] (9.2) paragraph 4:

A brace-or-equal-initializer shall appear only in the declaration of a data member. (For static data members, see 9.4.2; for non-static data members, see 12.6.2 **and 8.5.1**). A brace-or-equal-initializer for a non-static data member **specifies a default member initializer for the member, and** shall not directly or indirectly cause the implicit definition of a defaulted default constructor for the enclosing class or the exception specification of that constructor.

Change in [class.union] (9.5) paragraph 8:

A union-like class is a union or a class that has an anonymous union as a direct member. A union-like class X has a set of variant members. If X is a union, a non-static data member of X that is not an anonymous union is a variant member of X. In addition, a non-static data member of an anonymous union that is a member of X is also a variant member of X. At most one variant member of a union may have a **brace-or-equal-initializer** **default member initializer**.

Change in [class.ctor] (12.1) paragraph 4 bullet 2:

- any non-static data member with no **brace-or-equal-initializer** **default member initializer** is of reference type,

Change in [class.ctor] (12.1) paragraph 4 bullet 9:

- no non-static data member of its class has a **brace-or-equal-initializer** **default member initializer**, and

Change in [class.base.init] (12.6.2) paragraph 9 bullet 1:

- if the entity is a non-static data member that has a **brace-or-equal-initializer** **default member initializer (9.2)** and either
 - the constructor's class is a union (9.5), and no other variant member of that union is designated by a mem-initializer-id or the constructor's class is not a union, and, if the entity is a member of an anonymous union, no other member of that union is designated by a mem-initializer-id, the entity is initialized **from its default member initializer** as specified in 8.5;

Change in [class.base.init] (12.6.2) paragraph 10:

If a given non-static data member has both a **brace-or-equal-initializer** **default member initializer** and a mem-initializer, the initialization specified by the mem-initializer is performed, and the non-static data member's **brace-or-equal-initializer** **default member initializer** is ignored. [*Example:* Given

```
struct A {  
    int i = /* some integer expression with side effects */ ; A(int arg) : i(arg) { }  
    // ...  
};
```

the A(int) constructor will simply initialize i to the value of arg, and the side effects in i's **brace-or-equal-initializer** **default member initializer** will not take place. — *end example*]

Change in [class.base.init] (12.6.2) paragraph 11:

A temporary expression bound to a reference member from a **brace-or-equal-initializer** **default member initializer** is ill-formed.

Change in [class.base.init] (12.7) paragraph 5:

[...] When typeid is used in a constructor (including the mem-initializer or **brace-or-equal-initializer** **default member initializer** for a non-static data member) or in a destructor, [...]

Change in [class.base.init] (12.7) paragraph 6:

[...] When a dynamic_cast is used in a constructor (including the mem-initializer or **brace-or-equal-initializer** **default member initializer** for a non-static data member) or in a destructor, [...]

Change in [class.copy] (12.8) paragraph 15:

The implicitly-defined copy/move constructor for a non-union class X performs a memberwise copy/move of its bases and members. [*Note:* **brace-or-equal-initializer** **default member initializer**s of non-static data members are

ignored. See also the example in 12.6.2. — *end note*]

Ensure that an index entry for "default member initializers" is added, pointing to the definition in [class.mem] (9.2) paragraph 4.